



System Design Masterclass: Twitter

Handling 400 Billion Events at Massive Scale

Intent: Architect-Level Deep Dive
Architecture: Microservices & Polyglot Persistence
Focus: High-Throughput Read Paths

Global Footprint



United_States: 76.9M

Japan: 58.9M

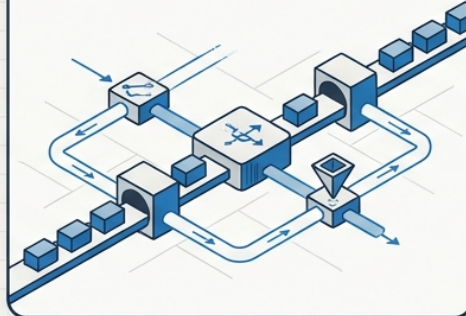
India: 23.6M

Geographic distribution drives capacity planning and edge caching.

Daily Throughput

400,000,000,000

Real-time events processed daily.



Storage Scale



Data hosted across tens of thousands of HDFS servers, compressed via LZO.

System Constraints Dashboard

Traffic Ratio



Write (1)

Read (1,000)

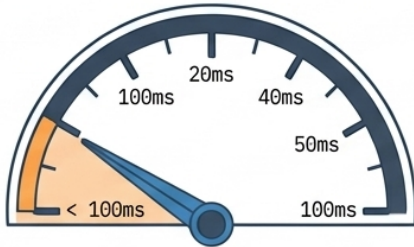
The architecture must optimize for extreme read-heavy workloads.

Consistency



Global timeline view accepts eventual consistency. The actor's local view demands immediate feedback.

Latency



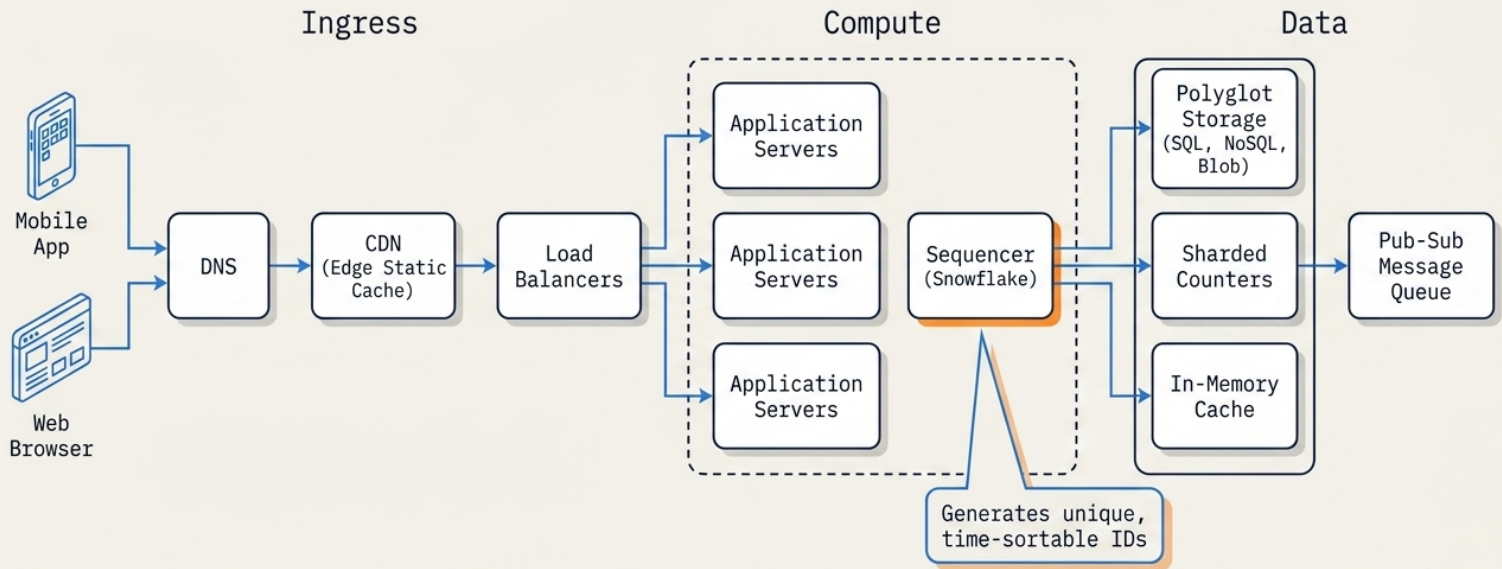
Sub-100ms response times for the read path; heavy feed generation pushed to asynchronous processing.

Availability



Uptime is critical; system serves as a global emergency communication channel.

The 10,000-Foot View: High-Level Architecture



WARN: Centralized LB becomes a bottleneck at massive scale. Client-Side LB evolution required.

API Contract Matrix: Payload Routing

POST /postTweet

```
{
  "user_id": 893,
  "content": "Hello",
  "media_field": "image"
}
```

Manhattan DB

Blobstore

GET /searchTweet

```
{
  "search_term": "#systemdesign",
  "exclude": "replies",
  "next_token": "abc12"
}
```

Results paginated
via next_token

Apache Lucene Index

GET /viewHome_timeline

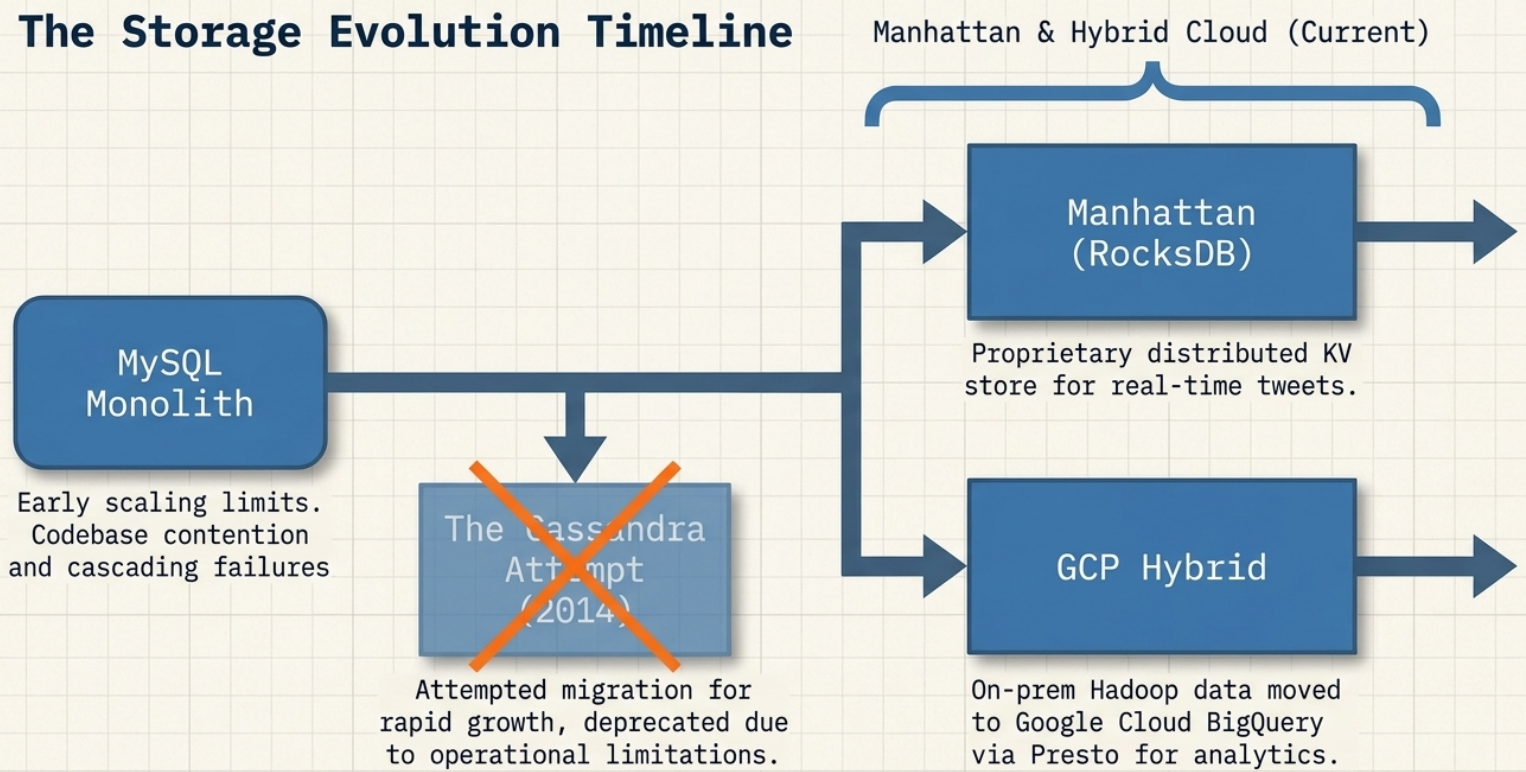
```
{
  "user_id": 893,
  "promoted_ads": true
}
```

Timeline Service

Cache

Sharded Counters

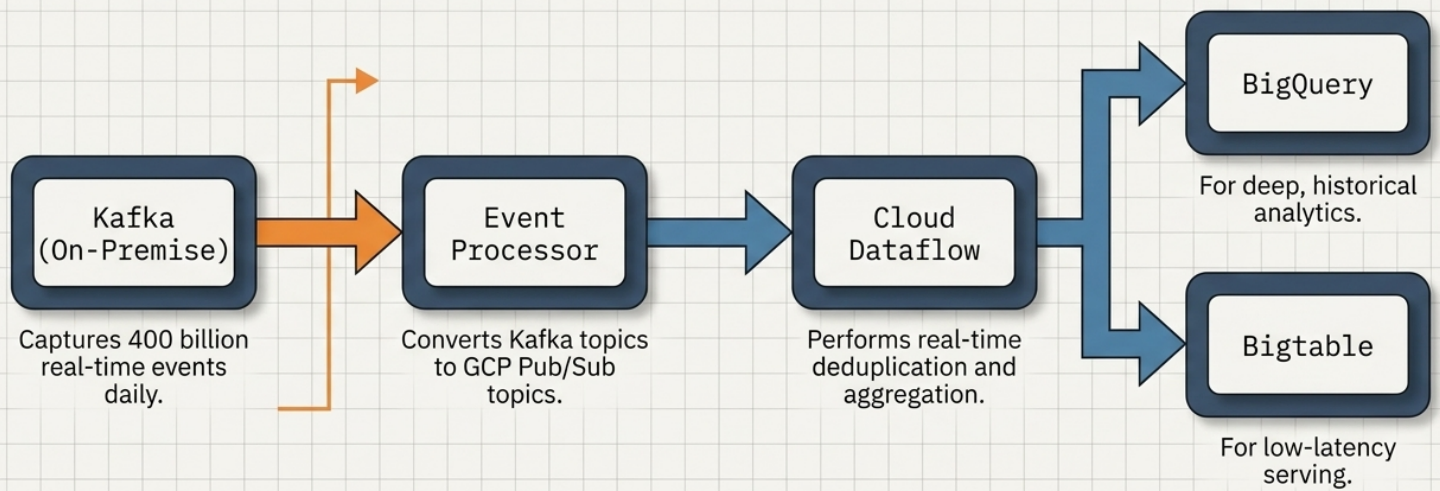
The Storage Evolution Timeline



Polyglot Persistence Diagnostic Table

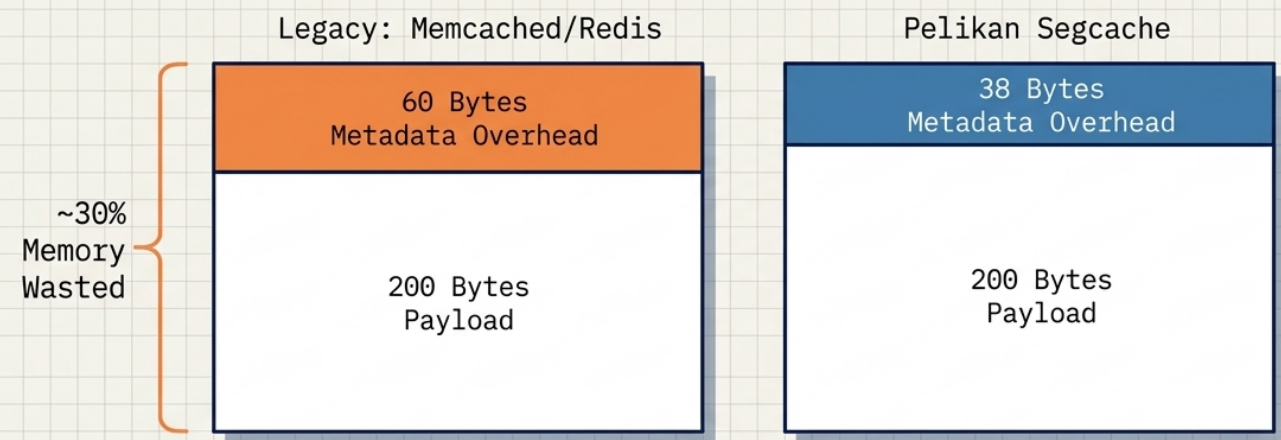
Technology	Data Model	Primary Use Case	Defining Feature
Manhattan	Key-Value (RocksDB)	Tweets, DMs, Accounts	Millions QPS, Heavy Read/Write scale
FlockDB	Graph	Social Graph (Followers)	Optimized for large adjacency lists & rapid traversal
SQL / Gizzard	Relational (Sharded)	Ad Exchanges, Campaigns	Requires strong consistency
Blobstore	Large Object	Photos, Videos	Durable storage & in-memory checkpoints
Apache Lucene	Inverted Index	Search	RAM for past 7 days (<100ms latency), Batch for history

Real-Time Analytical Pipeline (400B Events/Day)



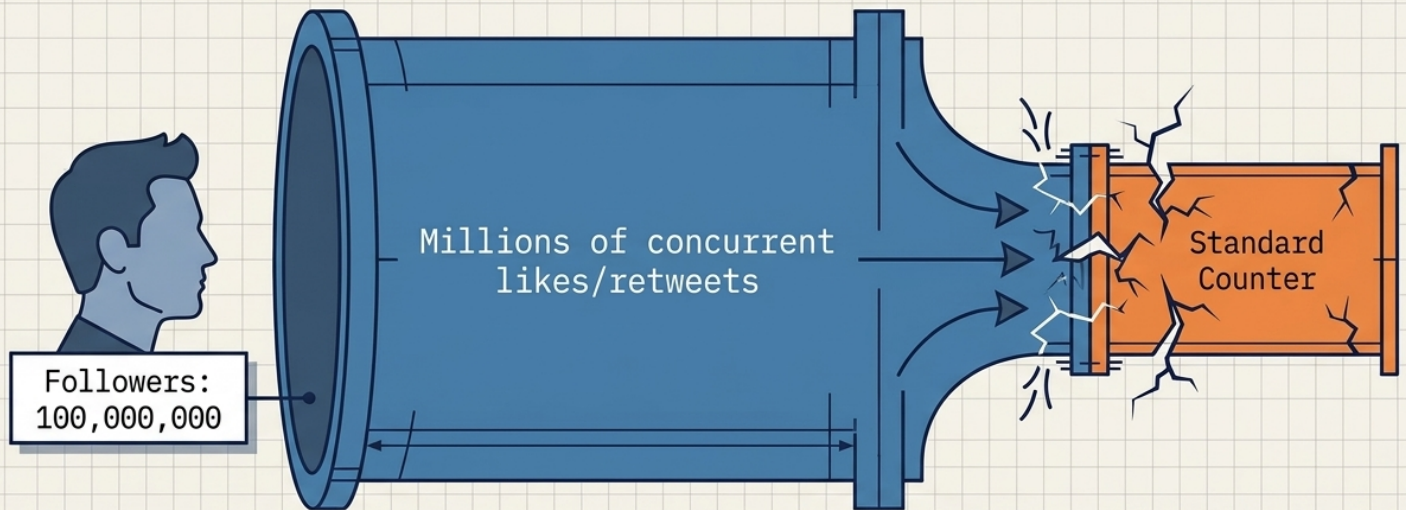
Cache Memory Efficiency: Pelikan Segcache

In massive-scale caching, small objects (200-300 bytes) dominate. Legacy metadata overhead wastes expensive RAM.



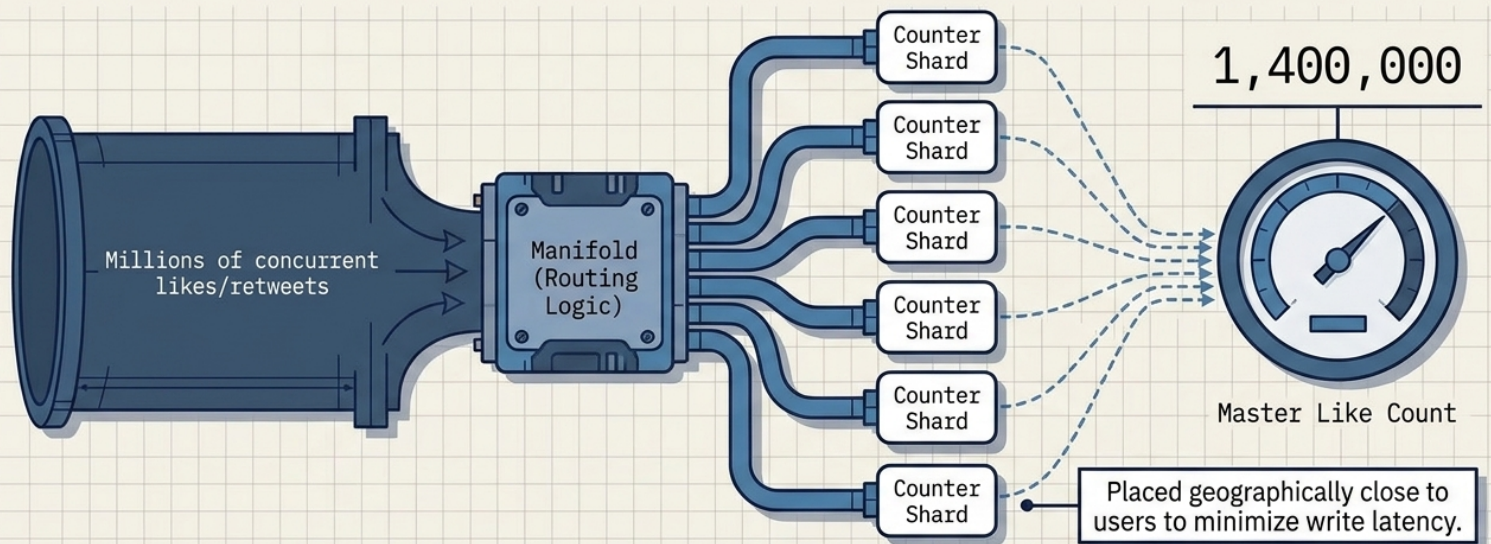
Takeaway: Segcache optimizes for high scalability, saving massive fleet-wide memory by shrinking per-object metadata.

The Heavy-Hitter Problem (Top-K Bottlenecks)



The Heavy-Hitter Problem: A single counter cannot handle the burst write throughput of viral content, leading to severe write contention and cascading failure.

Solving Write Contention: Sharded Counters



Application servers asynchronously aggregate geographically distributed shards to calculate global Top-K trends with eventual consistency.

Observability & Fleet Management

Distributed Tracing



Zipkin

Tracks request paths/latency across microservices.

To prevent massive overhead, it samples a percentage of requests, aggregates via Scribe, and stores in KV stores.

Dynamic Configuration



ZooKeeper

Provides coordination primitives: distributed locking, topology metadata, and leader election. Allows real-time config updates without cluster restarts.

Centralized Logging

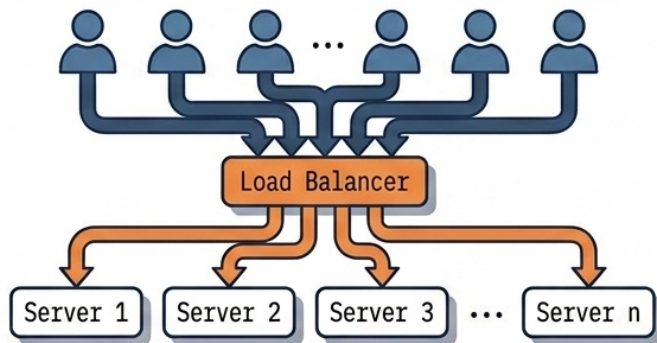


Splunk Enterprise

Replaced legacy Loglens as the central nervous system for visualizing and analyzing service health across the entire global fleet.

Load Balancing Evolution Matrix

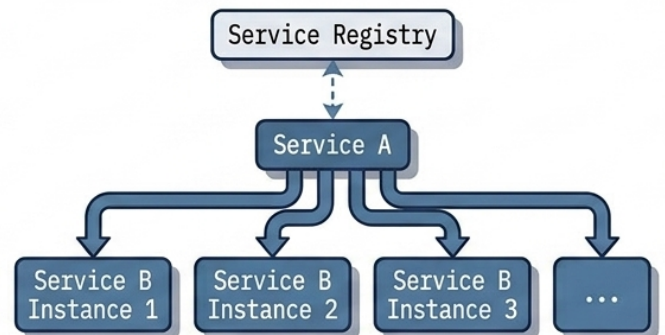
Centralized Load Balancing



Constraints

- Single point of failure
- Extra network hop (latency penalty)
- Throughput bandwidth bottleneck
- Central queue degrades Quality of Experience

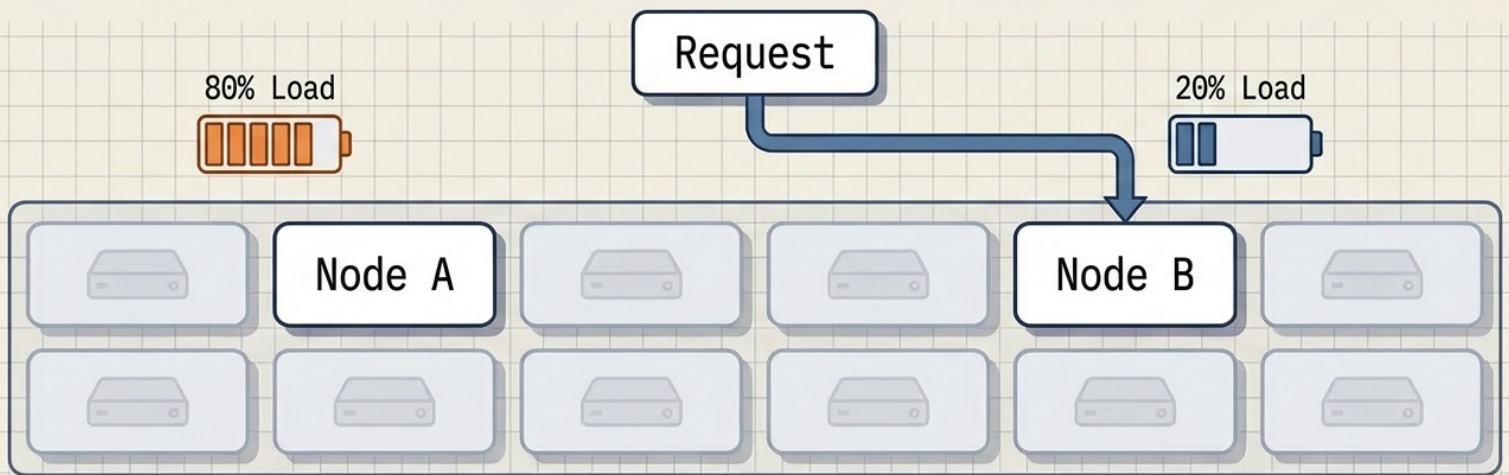
Client-Side LB (Finagle)



Advantages

- Reduced infrastructure overhead
- Zero intermediate hops
- No central bottleneck
- High resilience and decoupled routing

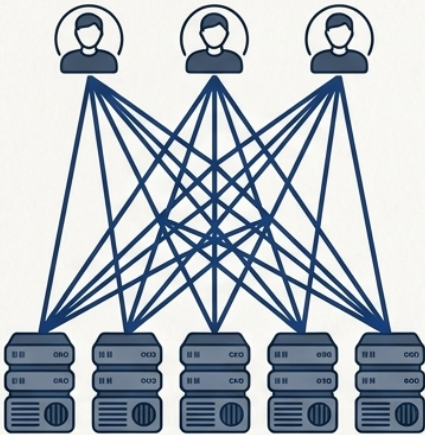
Routing Logic: The Power of Two Random Choices



P2C mathematics prove that comparing just two random nodes yields exponentially better load distribution than selecting one at random, while avoiding the massive computational overhead of checking every single node in the cluster.

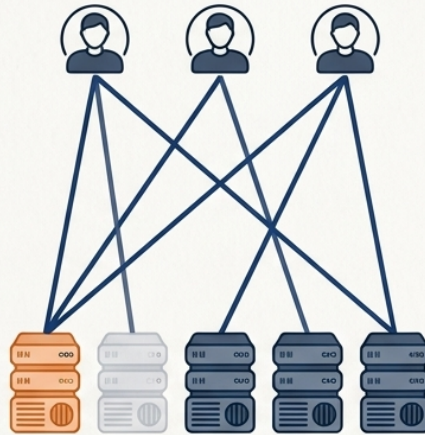
Diagnostic: Session Distribution Topologies (Layer 5)

Mesh Topology



Verdict: Fair distribution, but **catastrophic** coordination cost/overhead at scale.

Random Aperture



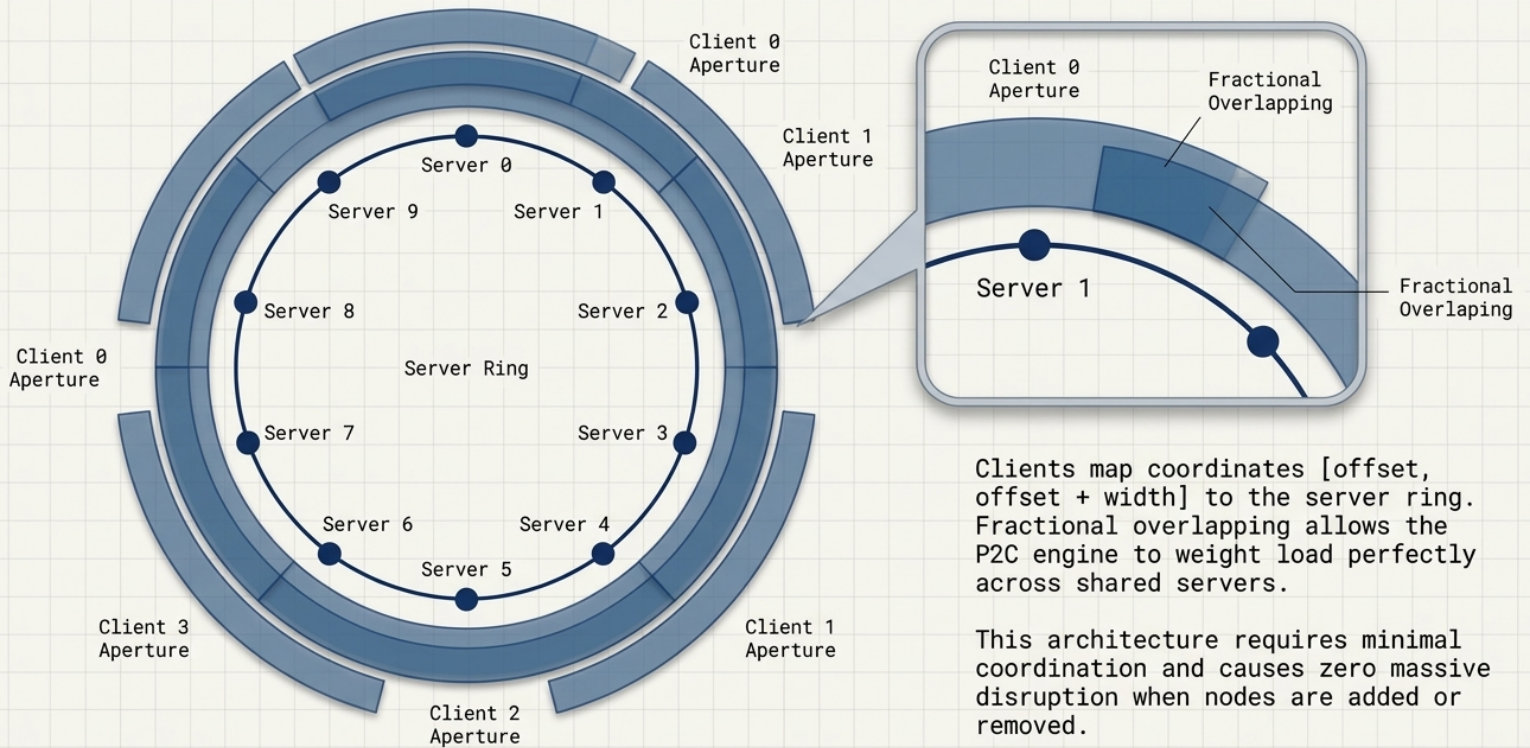
Verdict: Highly scalable, but creates unfair **"hot spots"** and **idle servers**.

Deterministic Aperture



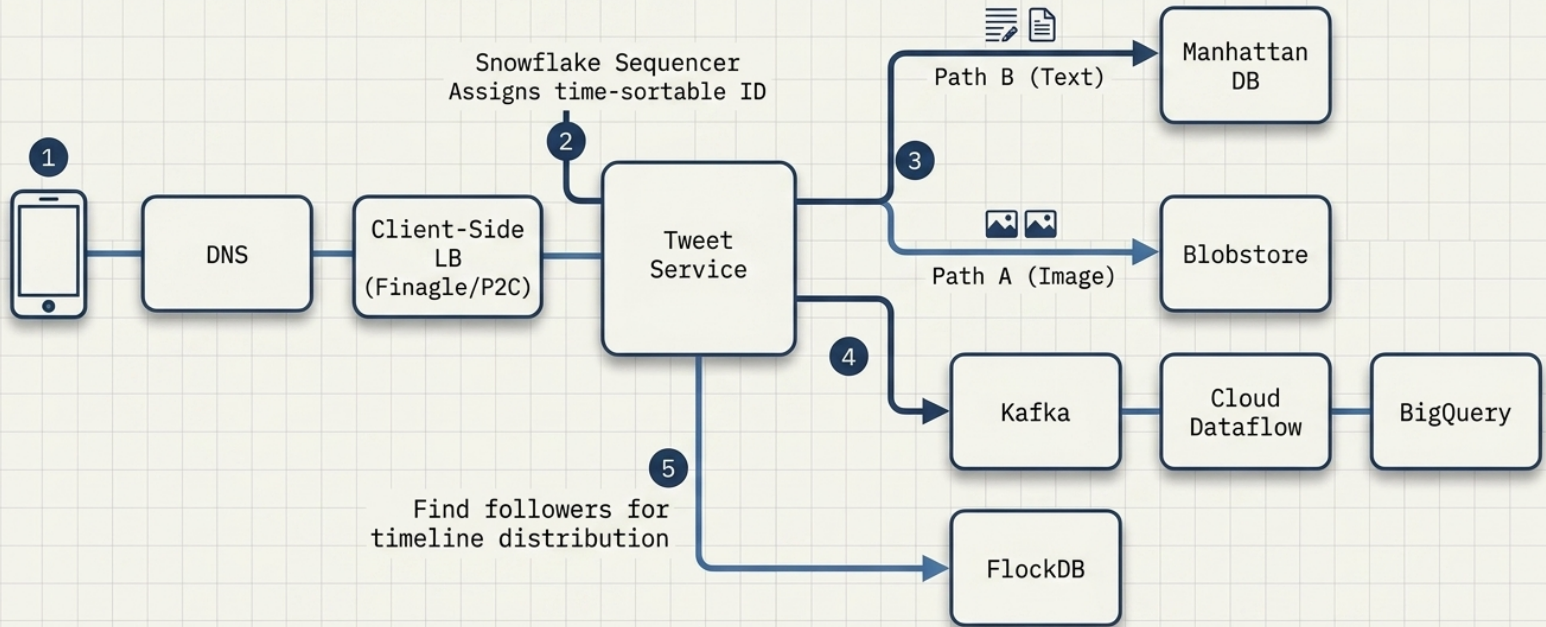
Verdict: The Finagle Solution. Balances scalability with **mathematical fairness** using fractional overlaps.

Deep Dive: Deterministic Aperture via Continuous Rings



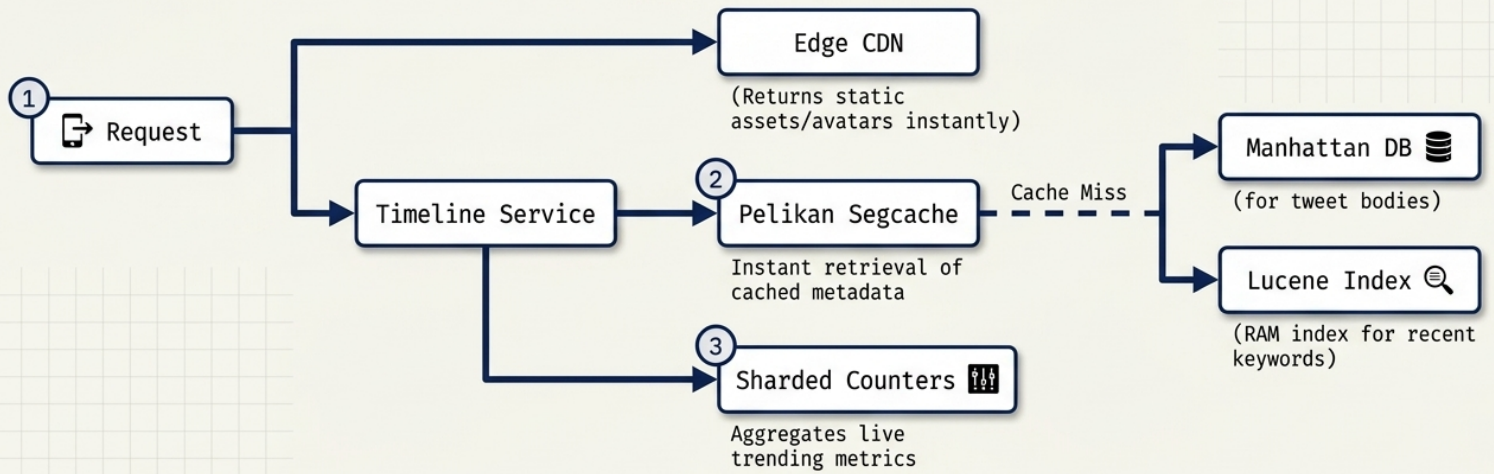
Synthesis: The Write Path

Scenario: A celebrity posts a tweet with an image attached.



Synthesis: The Read Path

Scenario: User requests /viewHome_timeline.



Result: Complex timeline rendered in <100ms via polyglot persistence, optimized caching, and client-side routing.